

树和二叉树

理论课程



厦門大學
XIAMEN UNIVERSITY



信息学院 黄 焯
(特色化示范性软件学院) 博士, 副教授
School of Informatics Wei Huang

内容要点

- 树的定义和基本术语
- 二叉树
- 遍历二叉树和线索二叉树
- 树和森林
- 哈夫曼树及其应用

目录

1	树的定义和基本术语
2	二叉树
3	遍历二叉树和线索二叉树
4	树和森林
5	哈夫曼树及其应用

基本概念

- 树(结构)：n个结点的有限集合($n \geq 0$)，
 - 空树：n=0的树(Tree)
- 根结点：没有前驱的结点
 - 对于非空树($n > 0$ 时)，有且只有1个根结点；
- 前驱和后继
 - 当 $n > 1$ 时，除根结点之外，树中的其它任意1个结点有且只有1个前驱；
 - 树中每个结点可以有m个后继($m \geq 0$)。

基本概念

- 树的递归定义

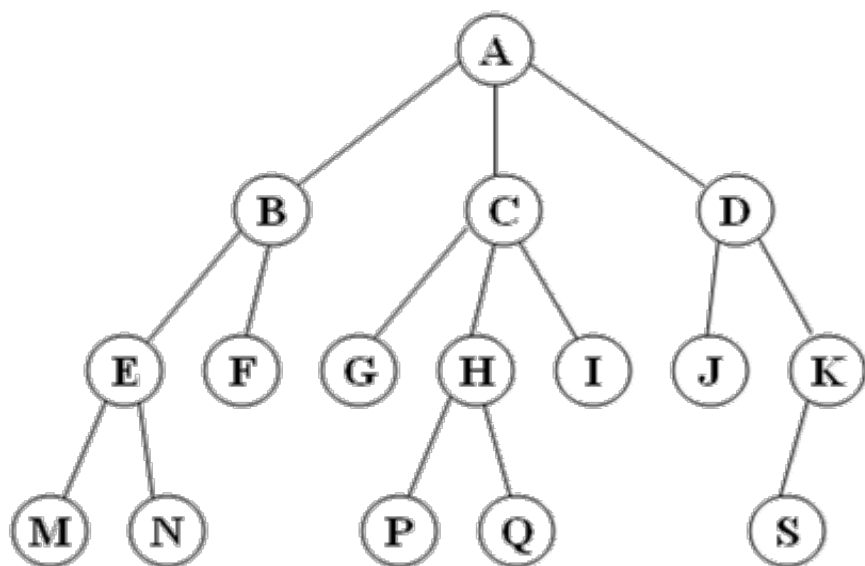
$$T = \begin{cases} \phi, & \text{当 } n=0 \text{ 时} \\ R, & \text{当 } n=1 \text{ 时} \\ R(T_1, \dots, T_m), & \text{当 } n>1 \text{ 时} \end{cases}$$

- 其中， $m>0$ ， R 表示根结点；

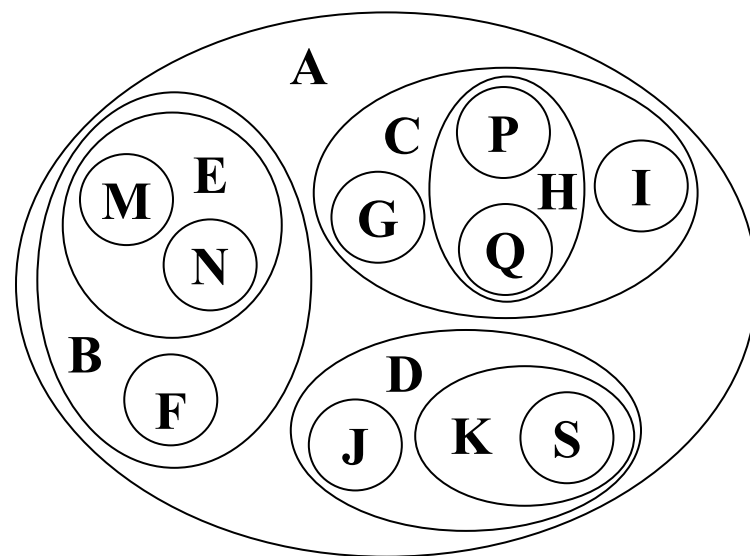
- $T_1, \dots, T_m$ 表示 m 棵子树（ m 个互不相交的有限集）。

树的表示

• 树的图形表示



树的嵌套集合形式表示

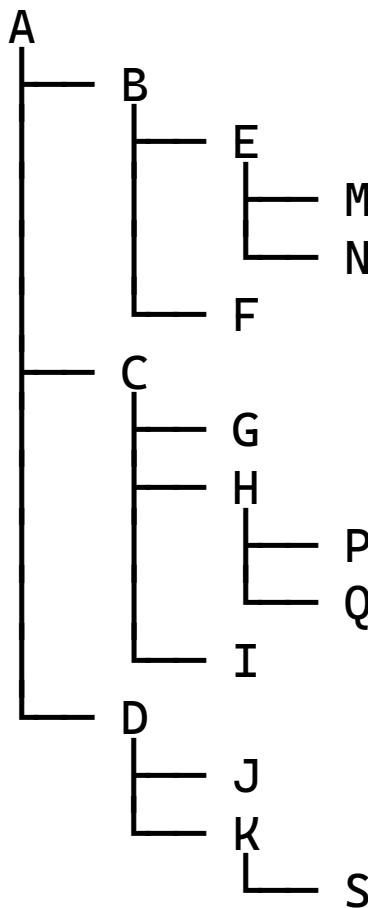


• 树的字符形式表示

$A(B(E(M, N), F), C(G, H(P, Q), I), D(J, K(S)))$;

树的表示

- 树的凹入表示



基本概念

- 树的结点：包含一个数据元素及若干个指针

- 如：ABCD...S

- 这些指针指向其拥有子树的根。

- 结点的度：结点拥有子树的个数。

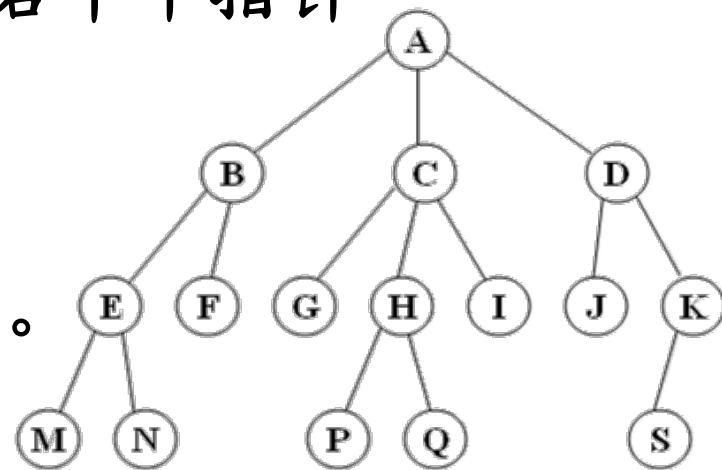
- C的度为3（孩子为G、H、I）。

- K的度为1（孩子为S）。

- F的度为0（没有孩子）。

- 叶子结点：度为0的结点即终端结点。

- 如：F、G、I、J、M、N、P、Q、S



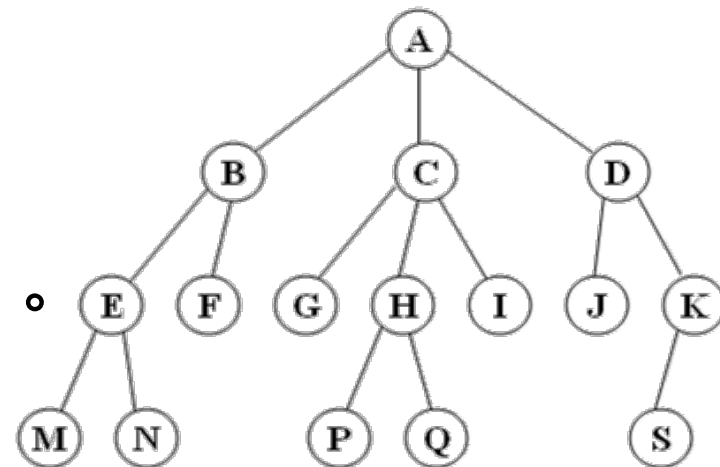
基本概念

- 分支结点：度 >0 的结点。

— 如：A、B、C、D、E、H、K。

- 树的度：树的度 $=\max(\text{结点的度})$ 。

— 如：结点 A、C 是 3，其余结点度均不超过 3，所以这棵树的度为 3。



- 结点的孩子：该结点指针指向的结点，结点拥有的（某棵）子树的根。

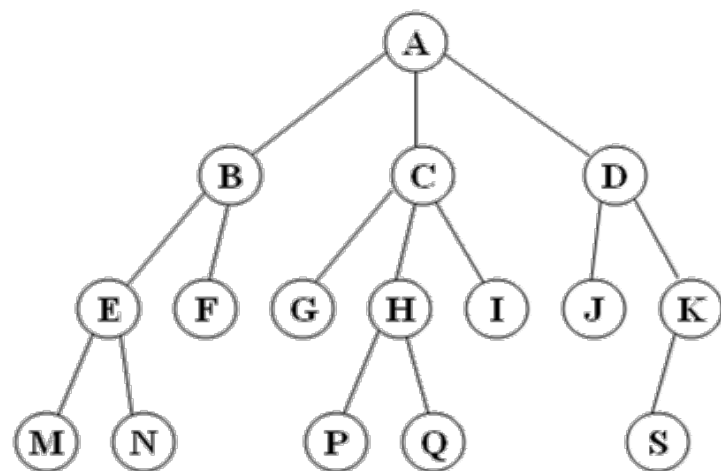
— 如：结点 B 的孩子是 E 和 F。结点 H 的孩子是 P 和 Q。结点 D 的孩子是 J 和 K。

基本概念

- 父亲结点：孩子结点的前驱（结点）。
 - 结点 E 和 F 的父亲结点是 B；结点 S 的父亲是 K；结点 A 是根结点，没有父亲。
- 兄弟结点：同一个父亲结点的孩子。
 - B、C、D 互为兄弟（父亲都是 A）；E 和 F 互为兄弟（父亲都是 B）；G、H、I 互为兄弟（父亲都是 C）；M 和 N 互为兄弟（父亲都是 E）；P 和 Q 互为兄弟（父亲都是 H）；J 和 K 互为兄弟（父亲都是 D）。
- 结点的层次：根结点定义为第1层；根结点的孩子定义为第2层；第 l 层结点的孩子定义为第 $l+1$ 层。

基本概念

- 树的高度(或深度) = $\max(\text{结点的层次})$ 。
 - 树的高度是4
- 说明：树的高度 $\neq$ 树的度(不同概念)。



有序树和无序树

- 有序树：(树中每一个)结点的各棵子树按一定的次序从左向右排列的树。
- 无序树：结点各子树与次序无关的树。
- 例如，对于有序树： $A(B, C) \neq A(C, B)$ ，
- 对于无序树： $A(B, C) = A(C, B)$ 。

森林

- 森林：m棵互不相交的树的有限集，
- 即 $F = \{ T_1, T_2, \dots, T_m \}$ ， $m \geq 0$ 。
- 当 $m=0$ 时，F称为空森林。
- 对于森林F，如果将每棵树都赋予同一个父亲结点R，即 $F = R(T_1, \dots, T_m)$ ，则森林成为一棵树。

树的两个基本特点

- 树的两个基本特点：
 - (1) 递归性质
 - (2) 层次结构
- 一个组织机构可以看成是一棵树。
 - 结点：机构名称及相关信息
 - 关系：<上级机构，下级机构>
- 树是一种非线性结构

树的层次结构特性

- 存储器上的目录结构是树型结构。

结点：逻辑盘、文件夹、文件

关系：
 <逻辑盘，文件夹>
 <文件夹，文件夹>
 <文件夹，文件>

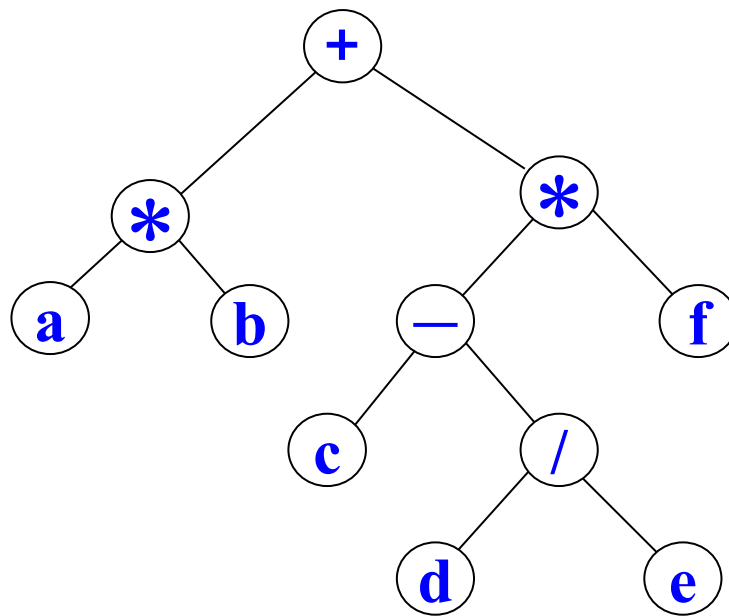
- 书目通常用树型结构描述。

结点：分类、书名、章节名称

关系：
 <分类，书名>
 <书名，章>
 <章，节>

树的层次结构特性

- 一个表达式可以表示成一棵树。
 - 结点：运算符、操作数；
 - 关系：运算规则。
- 可以将表达式 $a*b+(c-d/e)*f$ 表示成一棵树。



树的层次结构特性

- 树是一种非线性结构，具有递归性质和层次结构两种基本特点。
- 树的图形表示比较直观，常被用于算法的设计与分析，而树的字符形式表示易于存储，常被用于构建树的输入。

目录

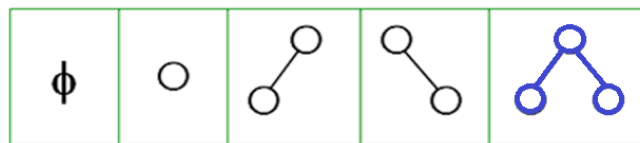
1	树的定义和基本术语
2	二叉树
3	遍历二叉树和线索二叉树
4	树和森林
5	哈夫曼树及其应用

二叉树的定义

- 二叉树：结点的度 ≤ 2 的(有序)树。

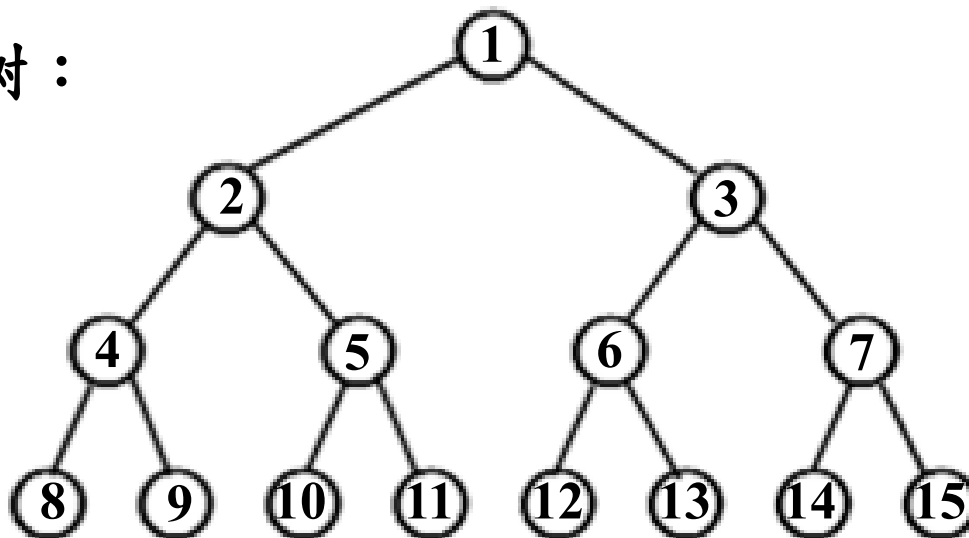
- 二叉树中每个结点的子树最多2棵。

- 二叉树的5种基本形态



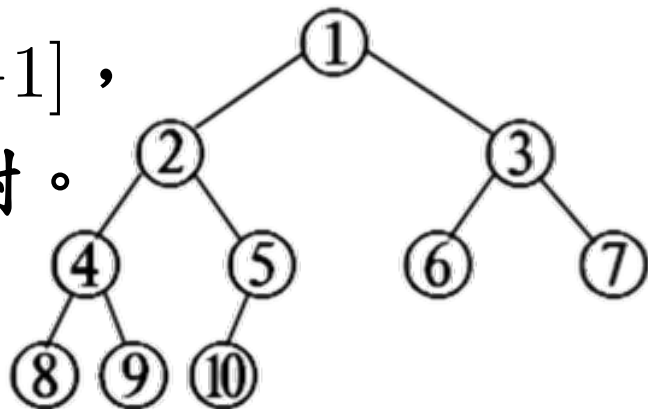
- 满二叉树：一棵高度为 k 且具有 $2^k - 1$ 个结点的二叉树。

- 如 $k=4$ 的满二叉树：



完全二叉树

- 完全二叉树
- 一棵高度为 k 、结点个数 $\in [2^{k-1}, 2^k - 1]$ ，且第 k 层结点都集中在左侧的二叉树。
- 满二叉树也是完全二叉树。



- 性质：
 - 在二叉树的第 i 层上最多有 2^{i-1} 个结点 ($i \geq 1$)。
 - 高度为 k 的二叉树最多有 $2^k - 1$ 个结点 ($k \geq 1$)。
 - 如果二叉树 T 的叶子结点数为 n_0 ，度为 2 的结点数为 n_2 ，则 $n_0 = n_2 + 1$ 。

二叉树的性质

- 如果二叉树 T 的叶子结点数为 n_0 ，度为2的结点数为 n_2 ，则 $n_0 = n_2 + 1$ 。
 - 证明：设二叉树 T 共有 n 个结点，叶子结点数为 n_0 ，度=1的结点数为 n_1 ，度=2的结点数为 n_2 ， T 的分支数为 m 。
 - (1) 由于二叉树中所有结点的度 ≤ 2 ，则有 $n = n_0 + n_1 + n_2$
 - (2) 除根结点外，其余结点都有唯一前驱，则有 $n - 1 = m$
 - (3) 由于度 = i 的结点有 i 个分支，则有 $m = 0 + 1 \times n_1 + 2 \times n_2$
- 即
$$n_0 = n_2 + 1$$

二叉树的性质

- 性质：具有 n 个结点的完全二叉树 T 高度为 $\lfloor \log_2 n \rfloor + 1$ 。

– 证明：设 T 的高度为 k 。

根据完全二叉树的定义可知，

T 的结点个数 $n \in [2^{k-1}, 2^k - 1]$ ，即

$$2^{k-1} \leq n \leq 2^k - 1 < 2^k$$

$$k-1 \leq \log_2 n < k$$

$$k-1 \leq \lfloor \log_2 n \rfloor \leq k-1$$

$$k = \lfloor \log_2 n \rfloor + 1, \text{ 证毕}$$

二叉树的性质

- 性质5：设完全二叉树含有 n 个结点。

- (1) 如果编号 $i=1$ ，则结点 i 是根结点；

- 如果编号 $i>1$ ，则结点 i 的父亲结点的编号是 $\lfloor i/2 \rfloor$ 。

- (2) 结点 i 的左孩子的编号为 $2i$ ；

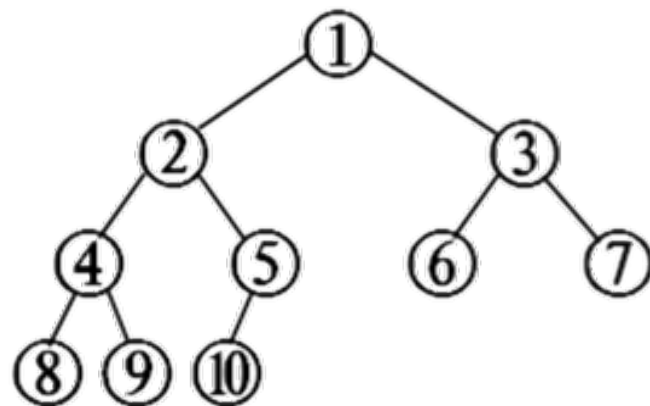
- 结点 i 的右孩子的编号为 $2i+1$ 。

- (3) 如果 $2i \leq n$ ，则结点 i 为分支结点；

- 如果 $2i > n$ ，则结点 i 为叶子结点。

- (4) 如果 n 为奇数，则每个分支结点都有左孩子和右孩子；

- 如果 n 为偶数，则编号为 $n/2$ 的分支结点(最大的分支结点)只有左孩子，没有右孩子。

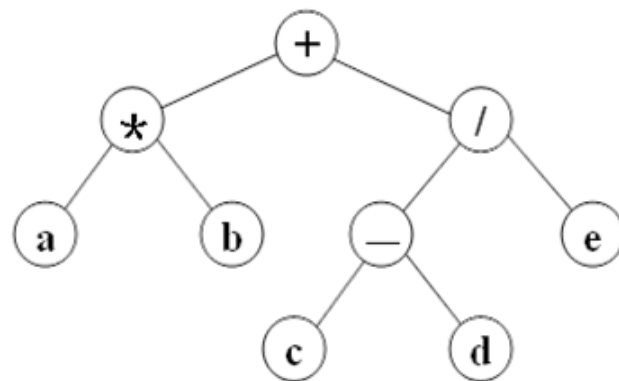


二叉树的定义

- 二叉树的顺序存储结构：

- 将二叉树的所有结点按照一定的次序存放的一组地址连续的存储单元中。
- 一般约定：从根结点开始，自上而下，自左而右排成一个线性序列。

- 例如，可以将下面的二叉树这样存



1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
+	*	/	a	b	-	e					c	d		

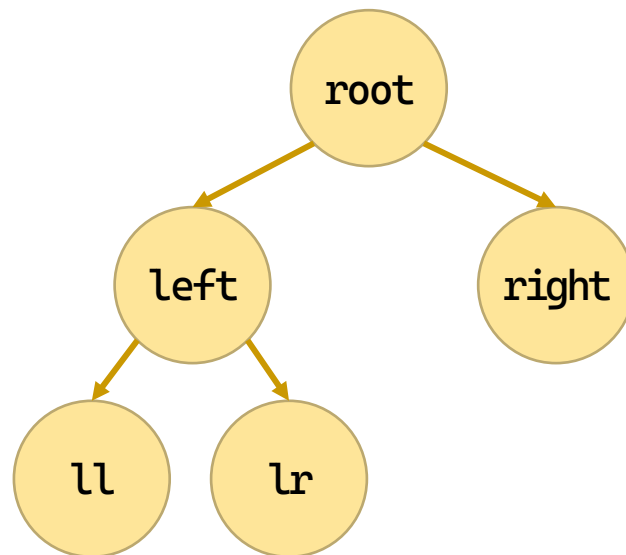
二叉树的定义

- 二叉树的顺序存储可以直接用顺序表描述
 - 将结点的编号和数组下标一一对应；
 - 结点的编号必须能反映其逻辑关系。
- 顺序存储结构适用于完全二叉树
 - 结点的编号容易和数组下标对应，且浪费的存储空间较小。

二叉树的定义

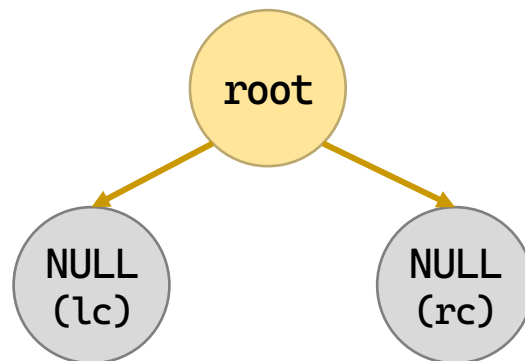
- 二叉树的存储结构

```
typedef struct Node {  
    char data;  
    struct Node* LChild;  
    struct Node* RChild;  
} BiTNode, * BiTree;
```



二叉树：初始化

- 初始化：只生成头指针。



```
int TreeInit(BiTree* T) {  
    *T = (BiTree)malloc(sizeof(BiTNode));  
    if (*T == NULL)  
        return 0;  
    // 初始化头结点：左右孩子置空，数据域可置为特殊值（如 '\0'）  
    (*T)->LChild = NULL;  
    (*T)->RChild = NULL;  
    (*T)->data = '\0';  
    return 1;  
}
```

// 头结点数据域不使用，置空
// 成功

二叉树：建立

- 扩展先序创建（递归）
 - 建结点，录入左子结点，录入右子结点

```
void create_bi_tree(BiTree* bt) {  
    char c;  
    c = getchar();  
    if (c == '.') {  
        (*bt) = NULL;  
    } else {  
        (*bt) = (BiTNode*)malloc(sizeof(BiTNode));  
        (*bt)->data = c;  
        create_bi_tree(&((*bt)->LChild));  
        create_bi_tree(&((*bt)->RChild));  
    }  
}
```

二叉树：建立

• 广义表创建（递归）

- “(”：入栈，入子左结点
- “)”：出栈，到父节点
- “,”：左结点入右结点
- 其它：建结点保存

```
if (c == '(') {  
    push(S, p); parent=p; left=0;  
}  
else if (c == ')') {  
    pop(S, &parent);  
}  
else if (c == ',') {  
    left = 1;  
}  
  
} else if (c >= 'A' && c <= 'Z') {  
    p=(BiTNode*)malloc(sizeof(BiTNode));  
    p->data=c; p->LChild=p->RChild=NULL;  
    if (*root == NULL)  
        *root = p;  
    else  
        if (left == 0)  
            parent->LChild = p;  
        else  
            parent->RChild = p;  
}
```

```

int create_bi_tree_bsl(BiTree* root, char* str) {
    SeqStack* S = (SeqStack*)malloc(sizeof(SeqStack));
    init_stack(S);
    BiTree p = NULL;
    BiTree parent = NULL;
    int left = 0;
    *root = NULL;
    for (; *str; str++) {
        char c = *str;
        if (c == '(') {
            push(S, p);
            parent = p;
            left = 0;
        } else if (c == ',') {
            left = 1;
        } else if (c == ')') {
            pop(S, &parent);
        }
    }
}

```

```

    } else if (c >= 'A' && c <= 'Z') {
        p = (BiTNode*)malloc(sizeof(BiTNode));
        p->data = c;
        p->LChild = p->RChild = NULL;
        if (*root == NULL) {
            *root = p;
        } else {
            if (left == 0)
                parent->LChild = p;
            else
                parent->RChild = p;
        }
    }
}

int empty = is_empty(S);
free(S);
return empty;
}

```

二叉树：显示

- 竖向树状打印

- 处理显示右子树（左转 90° 时，右在上）
- 显示缩进
- 处理显示左子树

```
void print_btree_vertical(BiTree root, int nLayer) {
    int i;
    if (root) {
        print_btree_vertical(root->RChild, nLayer + 1);
        for (i = 0; i < nLayer; i++)
            printf("    ");
        printf("%c \n", root->data);
        print_btree_vertical(root->LChild, nLayer + 1);
    }
}
```


二叉树：显示

- 广义表状打印

- 显示做左括号，处理左子树，显示逗号，处理右子树，显示右括号

```
void print_btree_bsl(BiTree bt) {  
    if (bt) {  
        printf("%c", bt->data);  
        if (bt->LChild || bt->RChild) {  
            printf("(");  
            print_btree_bsl(bt->LChild);  
            if (bt->RChild) {  
                printf(",");  
                print_btree_bsl(bt->RChild);  
            }  
            printf(")");  
        }  
    }  
}
```

目录

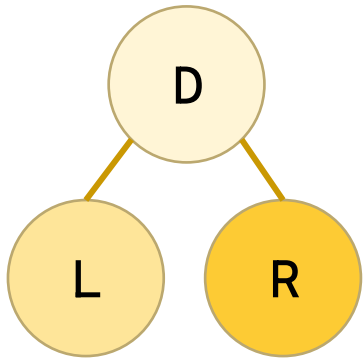
	3	遍历二叉树和线索二叉树
	3.1	遍历二叉树
	3.2	线索二叉树
	4	树和森林
	5	哈夫曼树及其应用

遍历二叉树

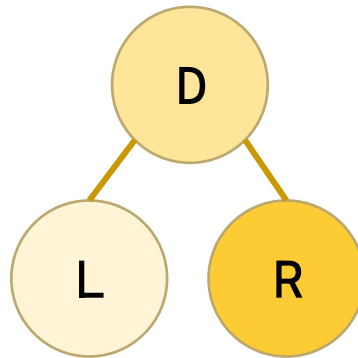
- 遍历二叉树：按照一定规则访问二叉树中的每个结点，使得每个结点均能被访问一次，且仅被访问一次。
- 通过一次遍历，可以使二叉树中的结点由非线性排列变为线性排列。遍历操作可以使二叉树线性化。
- 二叉树由3个基本单元组成：根结点、左子树和右子树，因此，遍历二叉树的问题可归结为解决：
 - (1) 访问根结点(D)；
 - (2) 遍历左子树(L)；
 - (3) 遍历右子树(R)。

遍历二叉树

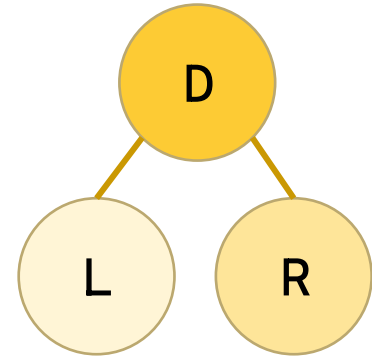
- 遍历二叉树共有六种方案：
 - DLR, LDR, LRD —— L先于R (常用)
 - DRL, RDL, RLD —— R先于L



前序遍历 DLR
PreOrder



中序遍历 LDR
InOrder



后序遍历 LRD
PostOrder

二叉树：递归版本

- 递归版本的遍历直接按照定义编写

```
void pre_order(BiTree root) {  
    if (root) {  
        visit(root);  
        pre_order(root->LChild);  
        pre_order(root->RChild);  
    }  
}
```

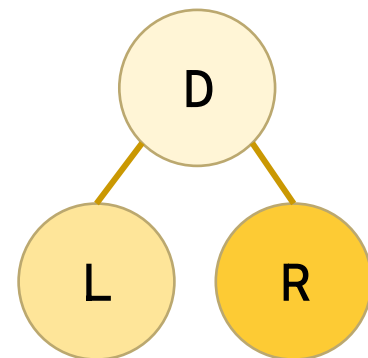
```
void post_order(BiTree root) {  
    if (root) {  
        post_order(root->LChild);  
        post_order(root->RChild);  
        visit(root);  
    }  
}
```

```
void in_order(BiTree root) {  
    if (root) {  
        in_order(root->LChild);  
        visit(root);  
        in_order(root->RChild);  
    }  
}
```

二叉树：栈版本

- 栈版本的前序遍历

- 访问根结点，压入根结点
- 如果有左结点，访问左结点
- 如果已无左结点，弹出父结点，访问右节点



- 实现方式

- 当前结点非空，访问、压入并进入左结点
- 否则，弹出栈，进入右节点

```
while (root || !is_empty(S)) {  
    if (root) {  
        visit(root);  
        push(S, root);  
        root = root->LChild;  
    } else {  
        pop(S, &root);  
        root = root->RChild;  
    }  
}
```

```

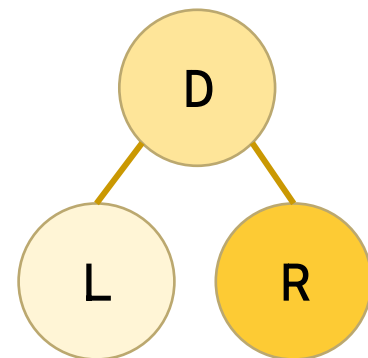
void pre_order_stack(BiTree root) {
    SeqStack* S = (SeqStack*)malloc(sizeof(SeqStack));
    init_stack(S);
    while (root || !is_empty(S)) {
        if (root) {
            visit(root);
            push(S, root);
            root = root->LChild;
        } else {
            pop(S, &root);
            root = root->RChild;
        }
    }
    free(S);
}

```

二叉树：栈版本

- 栈版本的前序遍历

- 不断压入左结点，进入最底层左结点
- 弹出结点，访问结点
- 进入右节点



```
while (root || !is_empty(S)) {  
    while (root) {  
        push(S, root);  
        root = root->LChild;  
    }  
    pop(S, &root);  
    visit(root);  
    root = root->RChild;  
}
```



```

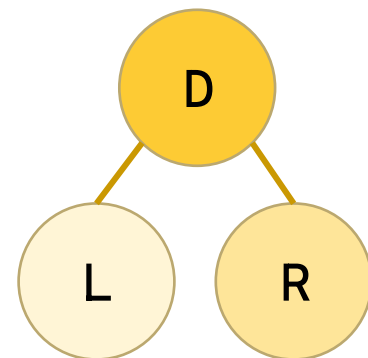
void in_order_stack(BiTree root) {
    SeqStack* S = (SeqStack*)malloc(sizeof(SeqStack));
    init_stack(S);
    while (root || !is_empty(S)) {
        while (root) {
            push(S, root);
            root = root->LChild;
        }
        pop(S, &root);
        visit(root);
        root = root->RChild;
    }
    free(S);
}

```

二叉树：栈版本

- 栈版本的后序遍历

- 不断压入，进入左结点（不访问）
- 访问栈顶，如果其有右结点右结点未访问，进入右节点
- 否则，访问当前结点，弹出结点，保存旧结点



```
while (root || !is_empty(S)) {
    if (root) {
        push(S, root);
        root = root->LChild;
    } else {
        get_top(S, &root);
        if (root->RChild == NULL
            || root->RChild == node) {
            visit(root);
            pop(S, &root);
            node = root;
            root = NULL;
        } else
            root = root->RChild;
    }
}
```

```

void post_order_stack(BiTree root) {
    BiTNode* node = root;
    SeqStack* S = (SeqStack*)malloc(sizeof(SeqStack));
    init_stack(S);
    while (root || !is_empty(S)) {
        if (root) {
            push(S, root);
            root = root->LChild;
        } else {
            get_top(S, &root);
            if (root->RChild == NULL || root->RChild == node) {
                visit(root);
                pop(S, &root);
                node = root;
                root = NULL;
            } else {
                root = root->RChild;
            }
        }
    }
    free(S);
}

```

二叉树遍历的应用

- 例：假设二叉树 $T = "+(* (a, b), /(- (c, d), e))"$ ，二叉链表 T 已建立。写出：前序、中序和后序遍历。

- 前序： $+*ab/-cde$

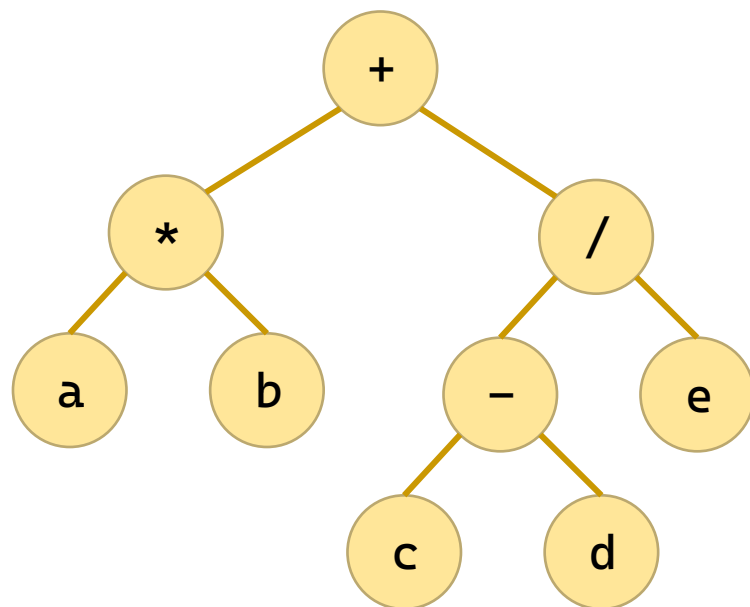
– $+(*ab, /(-cd, e)) \rightarrow +(*ab, /-cde)$

- 中序： $a*b+c-d/e$

– $+(a*b, /(c-d, e)) \rightarrow +(a*b, c-d/e)$

- 后序： $ab*cd-e/+$

– $+(ab*, /(cd-, e)) \rightarrow +(ab*, cd-e/+)$



二叉树遍历的应用

- **例3-1** 已知二叉树T的先序序列和中序序列分别为ABDFGCEH和BFDGACEH，试画出T。
 - 前序A最前，划分BFDG-A-CEH，即： $A(BDFG, CEH)$
 - 前序B最前，所以划分 ϕ -B-FDG，即： $B(, DFG)$
 - 前序D最前，所以划分 F-D-G，即： $D(F, G)$
 - 前序C最前，所以划分 ϕ -EH，即： $C(, EH)$
 - 前序E最前，所以划分 ϕ -E-H，即： $E(, H)$
- 综上， $T=A(B(, D(F, G)), C(, E(, H)))$

二叉树遍历的应用

- 例3-2 设二叉树T的中序序列和后序序列分别为：中序：3, 7, 11, 14, 18, 22, 27, 35、后序：3, 11, 7, 14, 27, 35, 22, 18，试画出二叉树T。
 - 后序18最后，划分3, 11, 7, 14-18-27, 35, 22
 - 后序14最后，所以划分3, 11, 7-14- ϕ
 - 后序7最后，所以划分 3-7-11
 - 后序22最后，所以划分 ϕ -22-27, 35
 - 后序35最后，所以划分 27-35- ϕ
- 综上， $T=18(14(7(3,11)),22(,35(27)))$

目录

	3	遍历二叉树和线索二叉树
	3.1	遍历二叉树
	3.2	线索二叉树
	4	树和森林
	5	哈夫曼树及其应用

线索二叉树

- 线索：在结点的空指针域中存放指向某次遍历得到的后继或前驱的指针。
- 在具有 n 个结点的二叉链表(二叉树)中，存在 $n+1$ 个空指针域。
- 后继线索(右线索)：在空右指针域中存放指向某次遍历得到的后继结点的指针。
- 前驱线索(左线索)：在空左指针域中存放指向某次遍历得到的前驱结点的指针。

线索二叉树

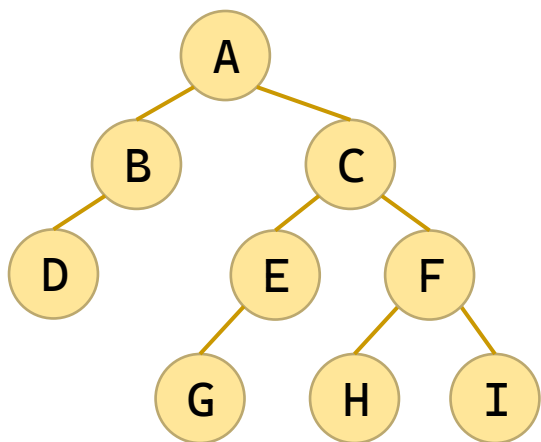
- 线索二叉树：根据某次遍历，在二叉树中的相关空指针域都写入线索(后继线索或前驱线索)，即成为线索二叉树。
- 线索二叉树可理解为已经线索化的二叉树。
- 先序后继：先序遍历中得到的后继(先序前驱, 中序后继, 中序前驱, 后序后继, 后序前驱)。

线索二叉树

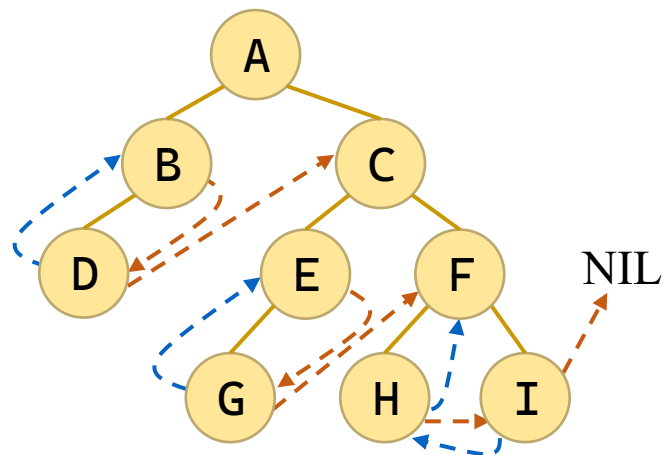
- 线索化原则：

- 先序后继：将右子树的根结点链入左子树的最后一个叶子结点。
- 中序后继：给定一个结点 x ，它是其左子树最后一个叶子结点的后继结点。
- 后序后继：
 - (1) 若结点 x 是二叉树的根，则其后继为空；
 - (2) 若结点 x 是其父亲的右孩子或是其父亲的左孩子且其父亲没有右孩子，则其后继即为父亲结点；
 - (3) 若结点 x 是其父亲的左孩子，且其父亲有右子树，则其后继为父亲的右子树上按后序遍历列出的第一个结点。

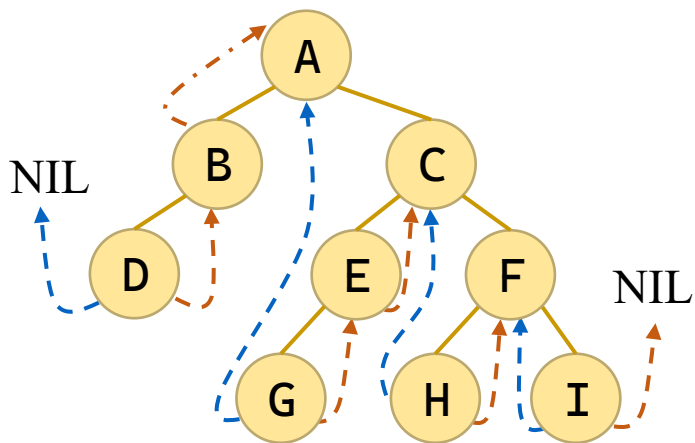
线索二叉树



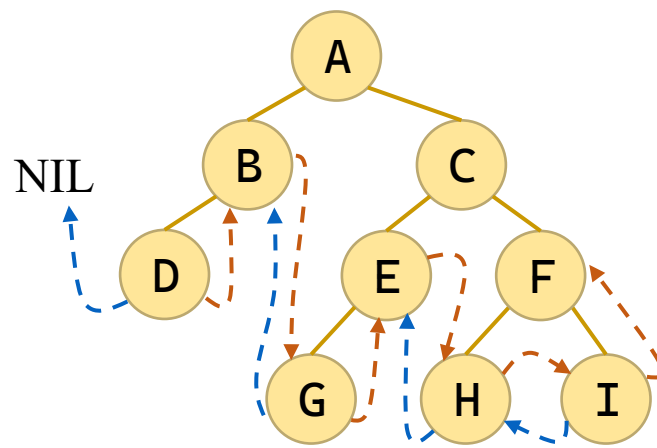
(a) 二叉树



(b) 先序线索树: ABDCEGFHI



(c) 中序线索树: DBAGECHFI



(d) 后序线索树: DBGEHIFCA

线索二叉树的定义

- 线索二叉树的存储结构

```
typedef struct Node {  
    char data;  
    struct Node* LChild;  
    struct Node* RChild;  
    int LTag;    // 0: 左孩子, 1: 左线索  
    int RTag;    // 0: 右孩子, 1: 右线索  
} BiTNode, * BiTree;  
typedef struct {  
    BiTree data[STACK_SIZE];  
    int top;  
} SeqStack;
```

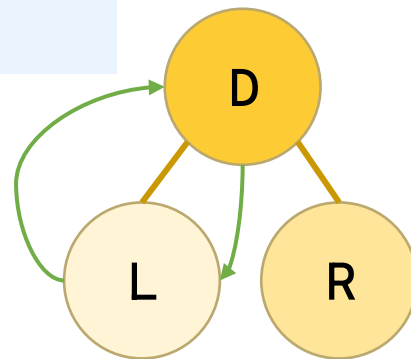
线索二叉树：中序建立线索

- 中序：线索化左子树，处理当前结点，线索化右子树

- 记录下前一结点
- 如果左子树空，
左结点指向前一结点
- 如果前一结点右子树空，
其右结点指向当前结点

```
*pre = p;
```

```
if (!p->LChild) {  
    p->LChild = *pre;  
    p->LTag = 1;  
}
```



```
if (*pre && !(*pre)->RChild) {  
    (*pre)->RChild = p;  
    (*pre)->RTag = 1;  
}
```

```

void InThread(BiTree p, BiTree* pre) {
    if (p) {
        InThread(p->LChild, pre);    // 1. 线索化左子树
        if (!p->LChild) {            // 2. 处理当前结点
            p->LChild = *pre; p->LTag = 1;
        }
        if (*pre && !(*pre)->RChild) {
            (*pre)->RChild = p; (*pre)->RTag = 1;
        }
        *pre = p;
        InThread(p->RChild, pre);    // 3. 线索化右子树
    }
}

void InThreading(BiTree* root) {    // 中序线索化入口
    BiTree pre = NULL;
    if (*root) {
        InThread(*root, &pre);
        if (pre && !pre->RChild)    // 处理最后一个结点的右线索
            pre->RTag = 1;          // 置为线索 (后继为空)
    }
}

```

线索二叉树：中序遍历

- 线索二叉树的中序遍历

- 遍历直至退出

- 找到最左子结点

- 先访问它

- 跳到其右节点（发挥索引作用）

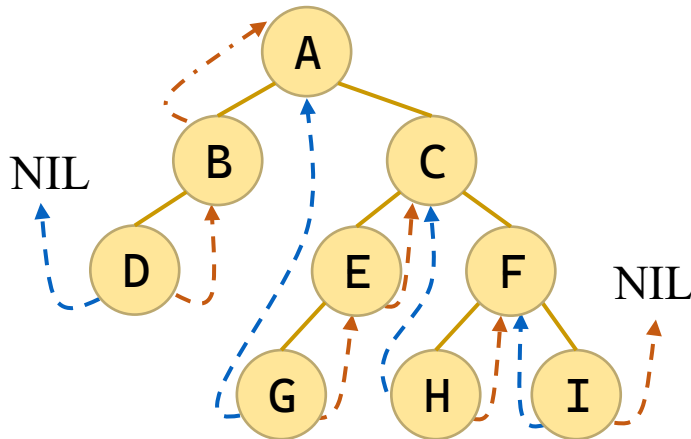
```
while (p) {
```

```
    while (p && p->LTag == 0)  
        p = p->LChild;
```

```
    visit(p);
```

```
    p = p->RChild;
```

```
}
```



线索二叉树：中序遍历

- 线索二叉树的中序遍历（优化）

- 找到最左子结点

- 退出时，为最左子的左（父）中结点

```
while (p && p->LTag == 0)
    p = p->LChild;
```

- 遍历直至退出

```
while (p) {
```

- 先访问它（中结点优先）

```
    visit(p);
```

- 跳到其右节点

```
    p = p->RChild;
```

- 如果是右子结点（而非线索），访问该右子节点的最左结点

```
    if (p->RTag == 0) {
        while (p && p->LTag == 0)
            p = p->LChild;
    }
```

```
}
```



```

void InOrderThreaded(BiTree root) {
    BiTree p = root;
    if (!p) return;
    // 先找到最左下结点
    while (p && p->LTag == 0)
        p = p->LChild;

    while (p) {
        visit(p);
        p = p->RChild;
        // 如果右指针不是线索, 到右子树的最左下结点
        if (p->RTag == 0) {
            p = p->RChild;
            while (p && p->LTag == 0)
                p = p->LChild;
        }
    }
}

```

目录

	2	二叉树
	3	遍历二叉树和线索二叉树
	4	树和森林
	5	哈夫曼树及其应用
	6	小结

树的存储结构

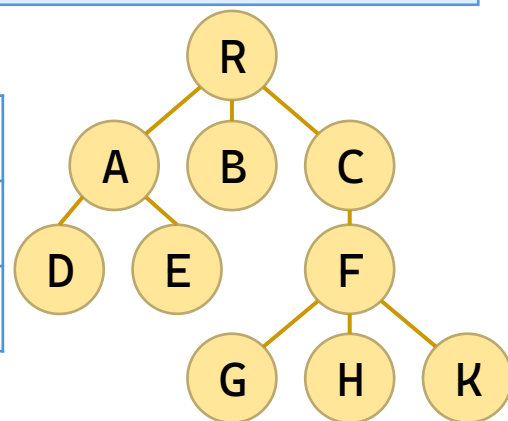
- 树的父亲表示法，用顺序表存储树的结点，每个结点包含数据域和指针域(父亲位置)。
- 树的孩子表示法，每个结点建立1个链表，用于存储该结点的所有孩子结点。
- 树的孩子兄弟表示法，用二叉树的存储结构表示树，第1个孩子为左孩子，其它兄弟依次存储为右孩子。

树的父亲表示法定义

• 树的父亲表示法存储结构

```
typedef struct {  
    char data;           // 数据域（可根据需要修改类型）  
    int parent;          // 父结点在数组中的位置（下标），根为 -1  
} PTNode;  
typedef struct {  
    PTNode nodes[MAX_TREE_SIZE]; // 结点数组  
    int n;                       // 当前结点个数  
} PTree;
```

位置	0	1	2	3	4	5	6	7	8	9
数据域	R	A	B	C	D	E	F	G	H	K
指针域	-1	0	0	0	1	1	3	6	6	6



树的儿子表示法定义

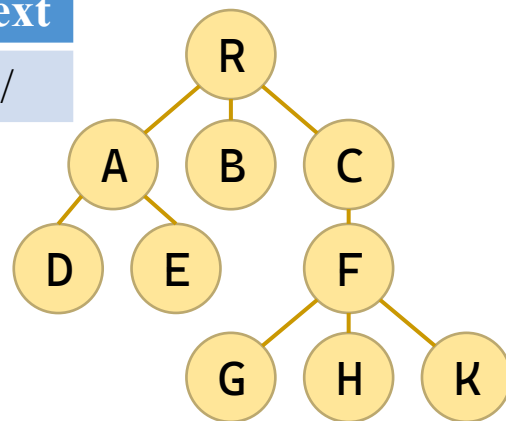
• 树的儿子表示法存储结构

```
typedef struct ChildNode {           // 孩子链表结点
    int child;                       // 孩子结点在数组中的下标
    struct ChildNode* next;          // 指向下一个孩子
} ChildNode;
typedef struct {                     // 树结点
    char data;                       // 数据域
    ChildNode* firstChild;           // 指向第一个孩子的链表结点
} CTNode;
typedef struct {                     // 树定义
    CTNode nodes[MAX_TREE_SIZE];    // 结点数组
    int n;                           // 结点个数
} CTree;
```

树的儿子表示法定义

- n : 6
- nodes : []

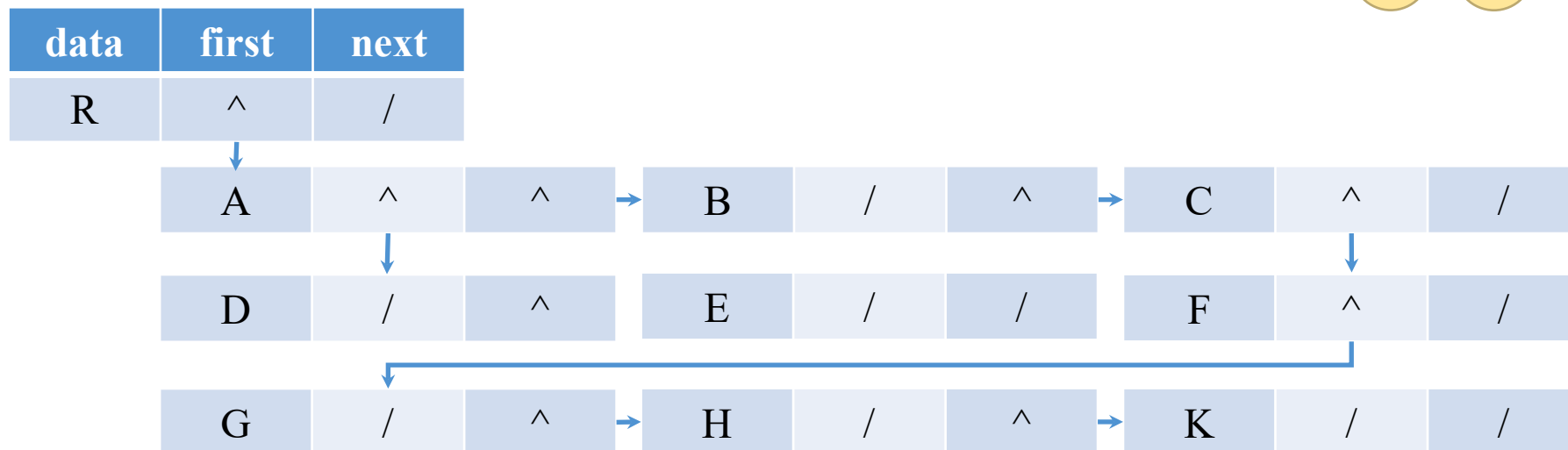
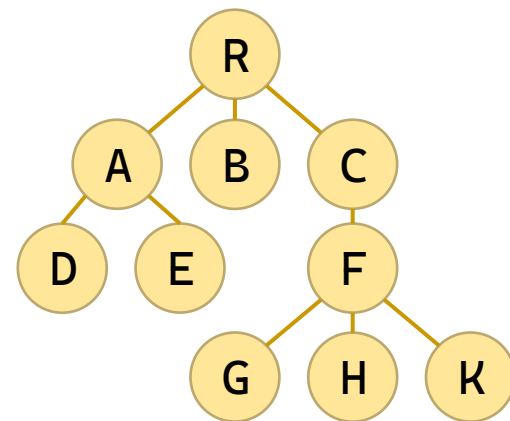
id	data	firstChd	child	next	child	next	child	next
0	R	^	1	^	2	^	3	/
1	A	^						
2	B	/						
3	C	^	6	/				
4	D	/						
5	E	/						
6	F	^	7	^	8	^	9	/
7	G	/						
8	H	/						
9	K	/						



树的兄弟表示法定义

• 树的兄弟表示法存储结构

```
typedef struct CSNode {  
    char data;                // 数据域  
    struct CSNode* firstChild; // 第一个孩子  
    struct CSNode* nextSibling; // 下一个兄弟  
} CSNode, *CSTree;
```



树转换为二叉树

- 树（尤其是多叉树）的存储结构比较复杂（如孩子链表、孩子兄弟等）。而“孩子兄弟表示法”本质上就是一种二叉树。将其转为标准的二叉树后，内存布局更规整。
- 树和森林没有统一的“中序遍历”定义，但二叉树有成熟且唯一的先序、中序、后序遍历算法。转换后，我们可以直接用二叉树的遍历算法去处理树和森林，无需为树重新设计复杂的遍历逻辑。

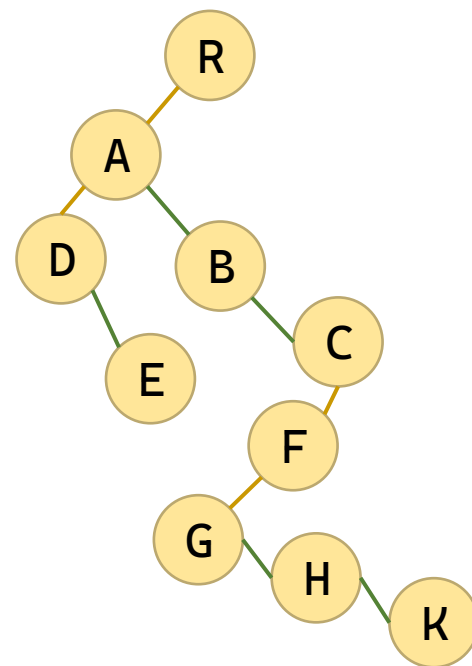
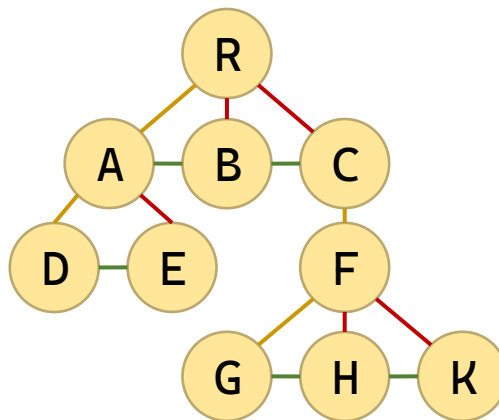
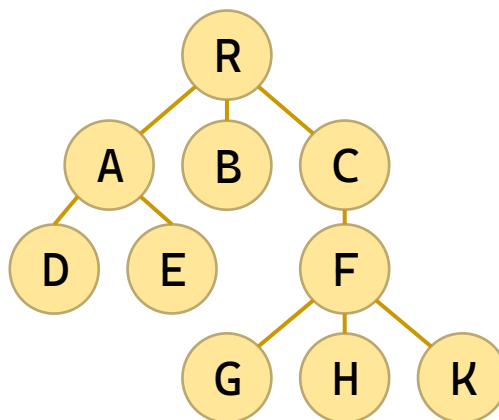
树转换为二叉树

- 由于二叉树和树都可用二叉链表作为存储结构，对比各自的结点结构可以看出，以二叉链表作为媒介可以导出树和二叉树之间的一个对应关系。
 - 从物理结构来看，树和二叉树的二叉链表是相同的，只是对指针的逻辑解释不同而已。
 - 从树的二叉链表表示的定义可知，任何一棵和树对应的二叉树，其右子树一定为空。
- 下图直观地展示了树和二叉树之间的对应关系。

树转换为二叉树

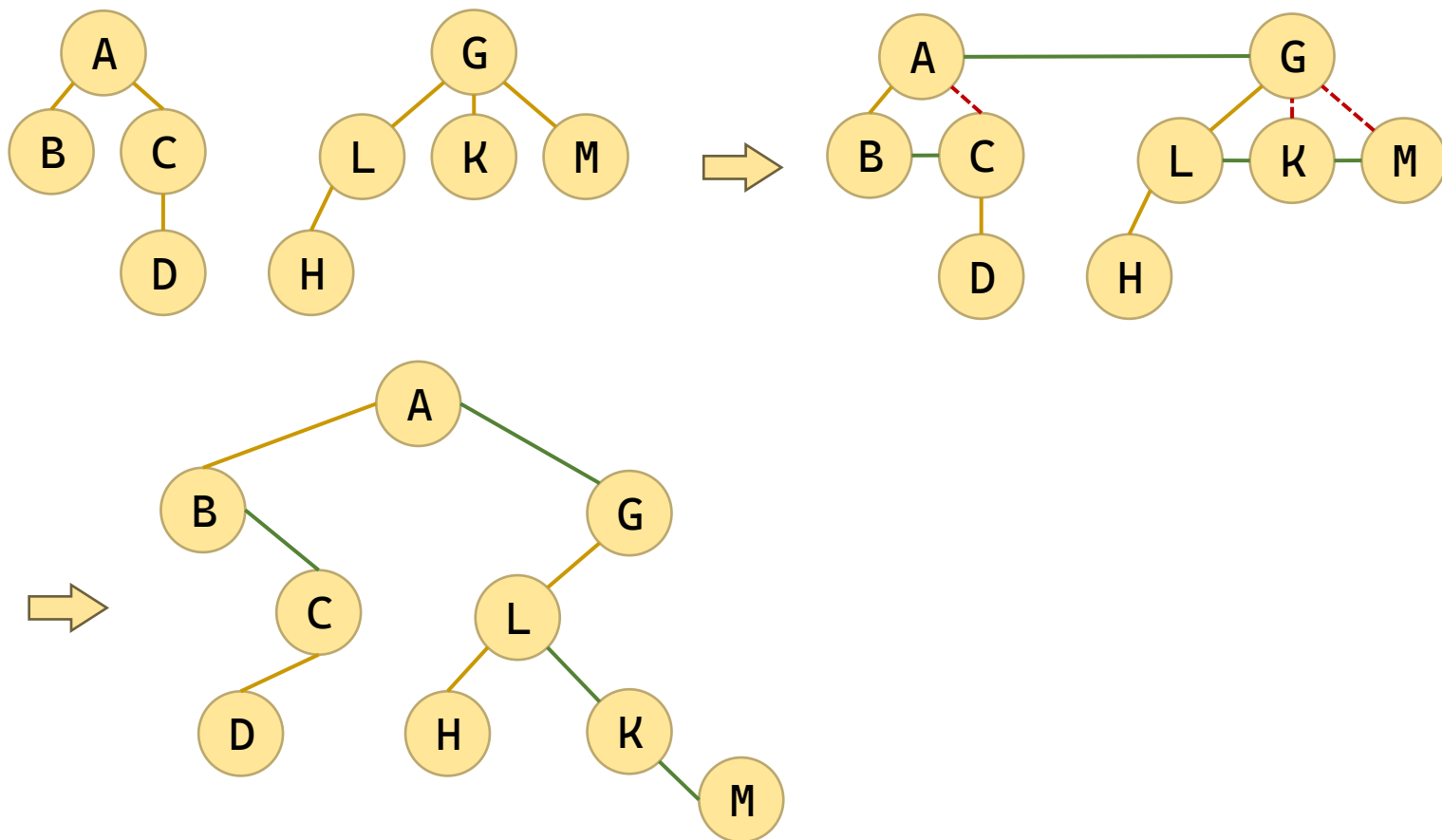
- 树与二叉树的转换

- 兄弟变右孩子
- 删除非长子
- 整理图形



森林转换为二叉树

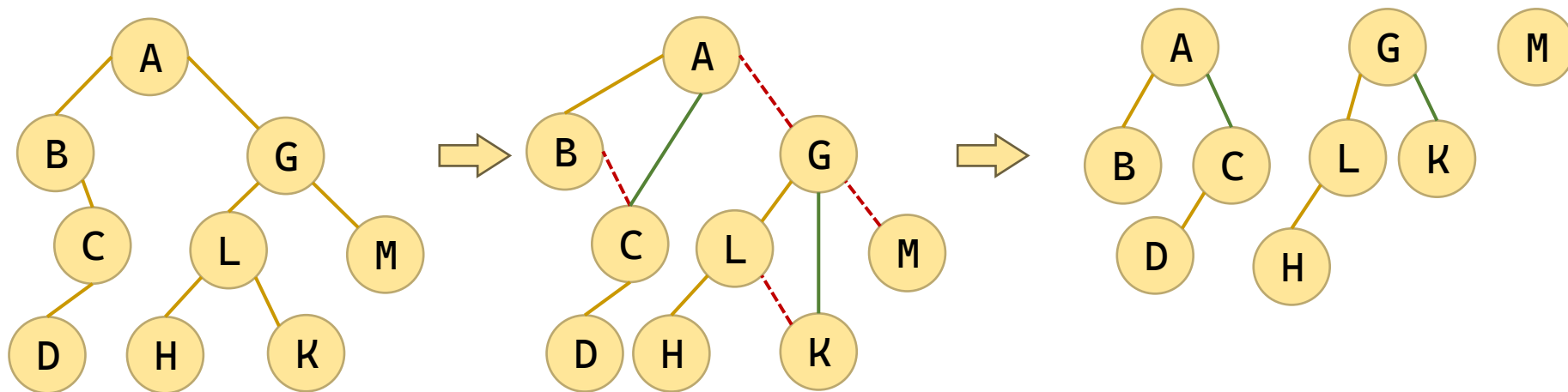
- 把森林中的树，相互视为兄弟



二叉树转换成森林

- 二叉树还原为森林

- 原“兄弟变右孩子”，改为“右孩子变兄弟”
- 原“删除非长子”，改为“删除右孩子”



树的遍历

- 树的先根遍历相当于其二叉树的前序遍历
- 树的后根遍历相当于其二叉树的中序遍历
- 树的层序遍历使用队列实现
 - 出队列，直到队列为空
 - 访问出队列的结点
 - 将其子结点（长子、兄弟结点）加入队列

```
void preOrderRoot(CSTree root) {  
    if (root == NULL) return;  
    printf("%c ", root->data);  
    preOrderRoot(root->firstChild);  
    preOrderRoot(root->nextSibling);  
}
```

```
void postOrderRoot(CSTree root) {  
    if (root == NULL) return;  
    postOrderRoot(root->firstChild);  
    printf("%c ", root->data);  
    postOrderRoot(root->nextSibling);  
}
```

```

void levelOrder(CSTree root) {
    if (root == NULL) return;
    Queue q;
    init_queue(&q);
    enqueue(&q, root);
    while (!is_empty(&q)) {
        CSTree cur = dequeue(&q);
        printf("%c ", cur->data);

        // 将当前结点的所有孩子依次入队
        CSTree child = cur->firstChild;
        while (child != NULL) {
            enqueue(&q, child);
            child = child->nextSibling;
        }
    }
}

```

树的应用

- 例3-6 采用树的儿子兄弟表示法描述组织机构。

子集树

- 所有的分支结点都含有相同个数的子树。
- 解决n个数，每个数有m中选择的组合问题。

遍历m叉子集树算法

```
for (i = 0; i < m; i++)  
    for (j = 0; j < m; j++)  
        for (k = 0; k < m; k++)  
            visit(path);
```

```
void dfs(int depth, int n, int m, int* path) {  
    if (depth == n) {  
        visit(path);  
        return;  
    }  
    for (int i = 0; i < m; i++) {  
        path[depth] = i;  
        dfs(depth + 1, n, m, path);  
    }  
}
```

n固定 (如: 3)

n不固定 (时间复杂度: $O(m^n)$)

子集树的应用：输出二进制

- 例3-7 设计算法，输出所有4位二进制数。

```
void SubTree(int k) {  
    if (k > n) {  
        for (int i = 1; i <= n; i++)  
            printf("%d", x[i]);  
        printf(" ");  
        return;  
    }  
    for (int i = 0; i < m; i++) {  
        x[k] = i;           // 第 k 位赋值为 i (0 或 1)  
        SubTree(k + 1);    // 递归处理下一位  
    }  
}
```

子集树的应用：n皇后问题

- 例3-8 n皇后问题找可行方案数的递归算法。
- 解决方法：增加一项，如果能放棋子则继续

```
void nQueen(int k) {  
    if (k == n) {  
        total++; return;  
    }  
    for (int col = 0; col < n; col++) {  
        if (isSafe(k, col)) {           // 如果第 k 行第 col 列安全  
            queenPos[k] = col;         // 放置皇后  
            nQueen(k + 1);              // 递归放置下一行  
        }  
    }  
}
```

```

int isSafe(int row, int col) {
    for (int i = 0; i < row; i++) {
        if (queenPos[i] == col) // 列冲突: 同一列
            return 0;
        if (abs(row - i) == abs(col - queenPos[i])) // 对角线冲突
            return 0;
    }
    return 1; // 安全
}

void nQueen(int k) {
    if (k == n) {
        total++; return;
    }
    for (int col = 0; col < n; col++) {
        if (isSafe(k, col)) { // 如果第 k 行第 col 列安全
            queenPos[k] = col; // 放置皇后
            nQueen(k + 1); // 递归放置下一行
        }
    }
}

```

子集树的应用：背包问题

- 例3-9 背包问题递归算法。
- 解决方法：增加一项，如果能放棋子则继续

```
void nQueen(int k) {  
    if (k == n) {  
        total++; return;  
    }  
    for (int col = 0; col < n; col++) {  
        if (isSafe(k, col)) {           // 如果第 k 行第 col 列安全  
            queenPos[k] = col;         // 放置皇后  
            nQueen(k + 1);             // 递归放置下一行  
        }  
    }  
}
```

```

void dfs(int depth, int n, int m, int* path) {
    // 1. 到达叶子结点 (所有物品决策完毕)
    if (depth == n) {
        // 更新最优解
        if (currentValue > bestValue) {
            bestValue = currentValue;
        }
        return;
    }
    // 2. 遍历 m 种状态 (m=2, 即 0:不选, 1:选)
    for (int i = 0; i < m; i++) {
        // 剪枝条件: 如果 i==1 (选), 但放不下当前物品, 则跳过该分支
        if (i == 1 && currentWeight + w[depth] > capacity) {
            continue; // 相当于这棵子树被剪掉了, 不往下递归
        }
        // ----- 做选择 -----
        path[depth] = i; // 记录当前决策 (完全符合模板)
        if (i == 1) { // 执行“选”的操作
            currentWeight += w[depth];
            currentValue += val[depth];
        }
    }
}

```

```

// 如果 i==0, 什么也不做 (不选)

// ----- 递归进入下一层 -----
dfs(depth + 1, n, m, path);

// ----- 回溯 (撤销选择) -----
if (i == 1) { // 撤销“选”的操作
    currentWeight -= w[depth];
    currentValue -= val[depth];
}
// 注意: path[depth] 不需要显式重置, 下一轮循环会直接覆盖
}
}

```

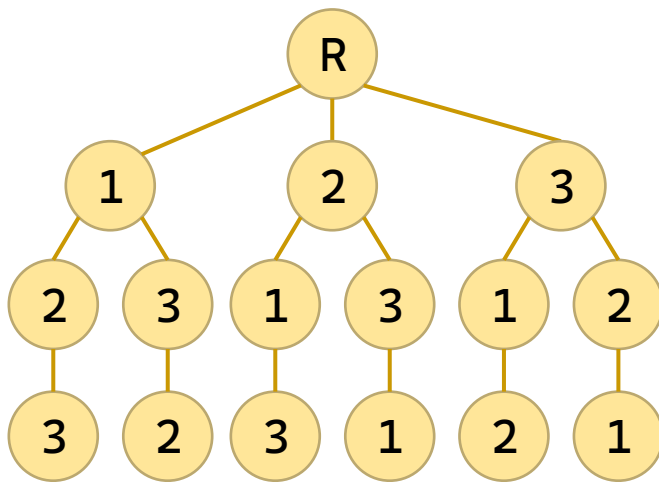
```

void knapSack(int k) {
    if (k == n) { // 1. 到达叶子结点 (所有物品决策完毕)
        if (currentValue > bestValue) // 当前方案价值最优
            bestValue = currentValue; // 则更新
        return;
    }
    // 2. 尝试“不选”第 k 个物品 (左分支)
    knapSack(k + 1);
    // 3. 尝试“选”第 k 个物品 (右分支)
    // 前提：加上当前物品后，重量不能超过背包容量
    if (currentWeight + w[k] <= capacity) {
        // 做选择：放入背包
        currentWeight += w[k];
        currentValue += v[k];
        // 递归处理下一个物品
        knapSack(k + 1);
        // 回溯：撤销选择，恢复现场 (这是必须的！)
        currentWeight -= w[k];
        currentValue -= v[k];
    }
}

```


排列树

- “排列树”是组合数学和算法设计中的一个概念，它指的是一种用于描述和解决排列问题的树形结构。
- 第 i 层分支结点拥有与其祖先都不同的 $n-i+1$ 棵子树。
- 解决 n 个数的全排列问题。



排列树

- 例3-10 遍历排列树算法(输出全排列)。

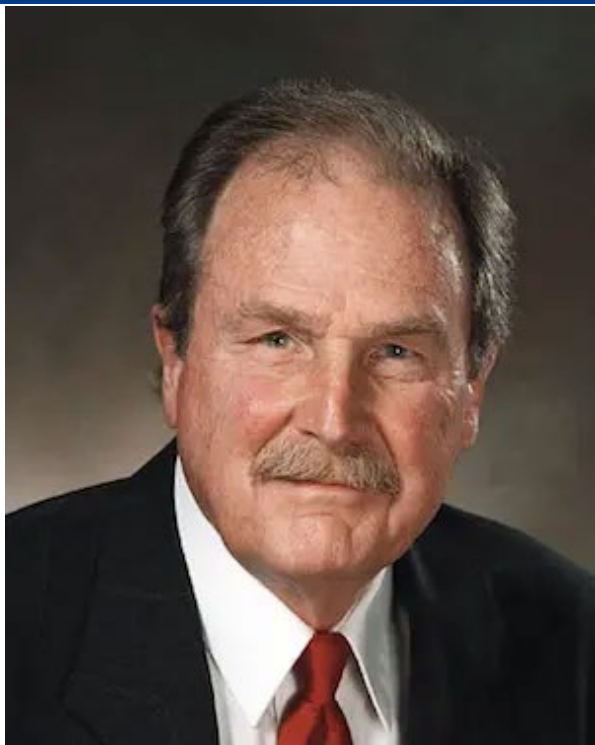
```
void backtrack(int pos) {  
    if (pos == n) {                // 所有位置已填满  
        process();  
        return;  
    }  
    // 尝试将 a[i] 放到 pos 位置 (i 从 pos 到 n-1)  
    for (int i = pos; i < n; i++) {  
        int temp = a[pos];        // 交换 a[pos] 和 a[i]  
        a[pos] = a[i];  
        a[i] = temp;  
        backtrack(pos + 1);        // 递归处理下一个位置  
        temp = a[pos];            // 回溯: 恢复交换  
        a[pos] = a[i];  
        a[i] = temp;  
    }  
}
```

目录

	2	二叉树
	3	遍历二叉树和线索二叉树
	4	树和森林
	5	哈夫曼树及其应用
	6	小结

David A. Huffman

1951年，哈夫曼和他在MIT信息论的同学需要选择是完成学期报告还是期末考试。导师Robert M. Fano给他们的学期报告的题目是，寻找最有效的二进制编码。由于无法证明哪个已有编码是最有效的，哈夫曼放弃对已有编码的研究，转向新的探索，最终发现了基于有序频率二叉树编码的想法，并很快证明了这个方法是最有效的。



1952年，David A. Huffman在麻省理工攻读博士时发表了《一种构建极小多余编码的方法》（A Method for the Construction of Minimum-Redundancy Codes）一文，它一般就叫做Huffman编码。

哈夫曼编码

- 前缀编码：一种不等长的字符编码，要求字符集中任何一个字符的编码都不是另一个字符编码的前缀。

例如，设字符集={ a, b, c, d, e }，则

a=00，b=01，c=10，d=110，e=111

是一种前缀编码，编成0001001011100110。

编码	00	01	00	10	111	00	110
消息	a	b	a	c	e	a	d

哈夫曼编码

- 哈夫曼编码：一种带权总量最小(如传送的二进制代码串最短)的前缀编码。
- 设计哈夫曼编码时应考虑：
 - (1) 数据对象，如字符集、符号集；
 - (2) 权值，如各个字符使用的频率；
 - (3) 哈夫曼树——用于带权总量最短；
 - (4) 前缀编码——使得能唯一译码。

哈夫曼编码

- 设计哈夫曼编码的一般步骤：

- (1)以权值集为叶子结点构造一颗哈夫曼树；
- (2)将哈夫曼树的左支标示0，右支标示1；
- (3)写出哈夫曼编码：从哈夫曼树根结点到各个叶子结点的路径代码。

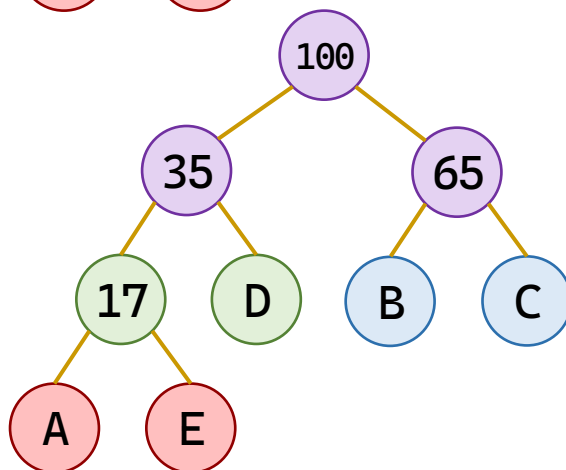
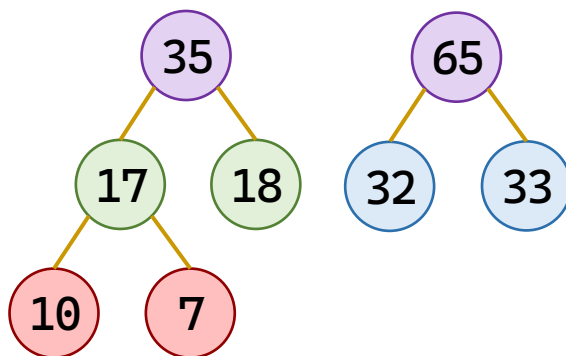
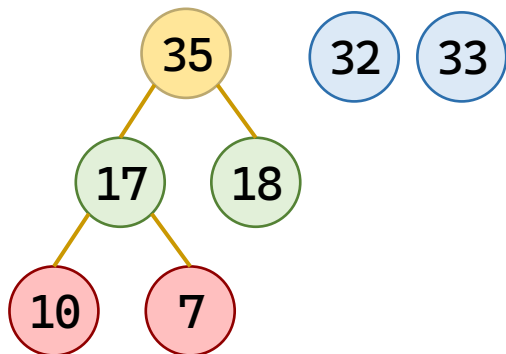
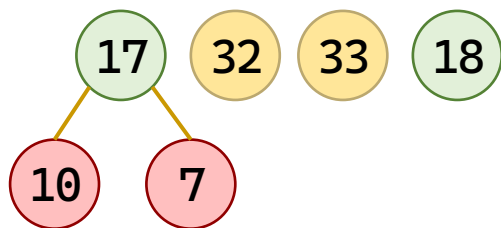
- 构造哈夫曼树

- 选择最轻的两堆合并
- 删掉合并前的两个权重，加上合并后的权重
- 剩下一堆时停止

把几堆不同重量的石子合并成一堆，每次只能合并两堆，搬石头的力气等于两堆重量之和，怎么合并最省力？

哈夫曼编码

- 例 已知字符集={A, B, C, D, E}，它们的权值={10, 32, 33, 18, 7}，试设计哈夫曼编码。



A	000
B	10
C	11
D	01
E	001

哈夫曼编码说明

- 几点说明：

- (1)在选取结点权值最小的两棵子树时,如果出现权值相同,可以任选其中一棵子树;
- (2)两棵结点权值最小的二叉树组成新的二叉树时,没有规定左右子树次序;
- (3)在哈夫曼树中,权值越大的叶子结点离根结点越近——哈夫曼树的应用依据;
- (4)具有 n 个叶子结点的哈夫曼树,共有 $2n-1$ 个结点(满足二叉树的性质)。

哈夫曼编码说明

– (4) 具有 n 个叶子结点的哈夫曼树，共有 $2n-1$ 个结点(满足二叉树的性质)。

∵ 在二叉树中，结点总数为 $n_0 + n_1 + n_2$ ，

在哈夫曼树中，没有度为1的结点，

即哈夫曼树的结点总数为 $n_0 + 0 + n_2$ ，

根据二叉树性质 $n_0 = n_2 + 1$ ，即 $n_2 = n_0 - 1$ ，

∴ 哈夫曼树的结点总数 $n_0 + n_2 = 2n_0 - 1 = 2n - 1$ 。

这个结论常用来在编程时为哈夫曼树静态分配数组空间，直接开 $2n$ 大小就够了。

哈夫曼树的实现

- 选择最轻的两堆合并

- 设最小值下标的初值为0
- 如果只考虑树根
- 如果最小值下标是初值0
- 或者找到更小
- 更新最小值
- 次小值下标多一个条件：
不等于最小值下标

```
int min_1 = 0, min_2 = 0;

for (i = 1; i <= range; i++)
    if (htree[i].parent == 0)

        if (min_1 == 0 ||

            htree[i].weight < htree[min_1].weight)

                min_1 = i;

*s1 = min_1;

if (htree[i].parent == 0 && i != *s1)
```

哈夫曼树的实现

- 删掉合并前的两个权重

- 将最小值和次小值的父结点指向新结点

```
htree[s1].parent = i;  
htree[s2].parent = i;
```

- 加上合并后的权重

- 更新新结点

```
htree[i].lchild = s1;  
htree[i].rchild = s2;  
htree[i].weight = htree[s1].weight +  
htree[s2].weight;
```

- 剩下一堆时停止

- 循环下标从 $n+1$ 到 $2n-1$ ，合并生成新节点

```
for (i = n + 1; i <= 2 * n - 1; i++)
```

```

#include <stdio.h>
#include <string.h>
#define MAX_LEAVES 20          // 最大叶子数（可调）
#define MAX_NODES (2 * MAX_LEAVES) // 最大总节点数
typedef struct {               // 哈夫曼树节点结构（静态三叉链表）
    int weight;                // 权值
    int parent;                // 双亲下标（0表示根）
    int lchild;                // 左孩子下标
    int rchild;                // 右孩子下标
} HNode;
// 全局变量
HNode htree[MAX_NODES];
int n; // 实际叶子个数

/* ===== 查找两个最小权值的根节点 ===== */
/*
* 功能：在 htree[1..range] 范围内，找出两个 parent==0 的节点，
*       分别作为最小权值 s1 和次小权值 s2。
* 算法：采用“打擂台”思想，用哨兵变量 min_1, min_2 记录下标，
*       一次遍历完成查找，避免先找最小再找次小的重复遍历。
*/

```

```

void selectMin(int range, int* s1, int* s2) {
    int i;
    int min_1 = 0;    // 最小值下标, 0表示未找到
    int min_2 = 0;    // 次小值下标
    for (i = 1; i <= range; i++)    // 第一趟: 找最小值
        if (htree[i].parent == 0)    // 只考虑森林中的树根
            if (min_1 == 0
                || htree[i].weight < htree[min_1].weight)
                min_1 = i;    // 发现更小的, 更新
    *s1 = min_1;    // 此时 min_1 一定不为0 (因为至少有两个根)

    for (i = 1; i <= range; i++) {    // 第二趟: 找次小值 (跳过 s1)
        if (htree[i].parent == 0 && i != *s1)
            if (min_2 == 0
                || htree[i].weight < htree[min_2].weight)
                min_2 = i;
    }
    *s2 = min_2;
}

/* ===== 构建哈夫曼树 ===== */
void createHuffmanTree(int weights[]) {
    int i;

```

```

int totalNodes = 2 * n - 1;    // 哈夫曼树总节点数
memset(htree, 0, sizeof(htree)); // 1. 初始化数组 (0下标不用)
for (i = 1; i <= n; i++)      // 2. 将前 n 个位置设为叶子节点
    htree[i].weight = weights[i - 1];
for (i = n + 1; i <= totalNodes; i++) { // 3. 合并生成新节点
    int s1, s2;
    selectMin(i - 1, &s1, &s2); // 在前i-1个节点中找最小
    htree[s1].parent = i; htree[s2].parent = i; // 父节点指向
    htree[i].lchild = s1; htree[i].rchild = s2;
    htree[i].weight = htree[s1].weight + htree[s2].weight;
    printf("第%2d次合并: 选[%d]权=%2d和[%d]权=%2d, 建[%d]权=%d\n",
        i - n, s1, htree[s1].weight, s2, htree[s2].weight,
        i, htree[i].weight);
}
// 打印数组存储表 (便于画树验证)
printf("\n===== 哈夫曼数组存储表 =====\n");
printf("下标\t权值\t父节点\t左孩子\t右孩子\n");
for (i = 1; i <= totalNodes; i++)
    printf("%d\t%d\t%d\t%d\t%d\n", i, htree[i].weight,
        htree[i].parent, htree[i].lchild, htree[i].rchild);
}

```

```

/* ===== 生成哈夫曼编码 ===== */
void getHuffmanCodes(char codes[][MAX_LEAVES]) {
    int i, cur, p, idx;
    char temp[MAX_LEAVES];
    for (i = 1; i <= n; i++) {    // 对每个叶子
        idx = 0; cur = i;
        while (htree[cur].parent != 0) {    // 从叶子向上爬到根
            p = htree[cur].parent;
            temp[idx++] = (htree[p].lchild == cur) ? '0' : '1';
            cur = p;
        }
        temp[idx] = '\0';
        int len = idx;
        for (int j = 0; j < len; j++)    // 叶子往上走的编码需要反转
            codes[i - 1][j] = temp[len - 1 - j];
        codes[i - 1][len] = '\0';
    }
    if (n == 1) {    // 特判：如果只有一个叶子，编码规定为 "0"
        codes[0][0] = '0'; codes[0][1] = '\0';
    }
}

```

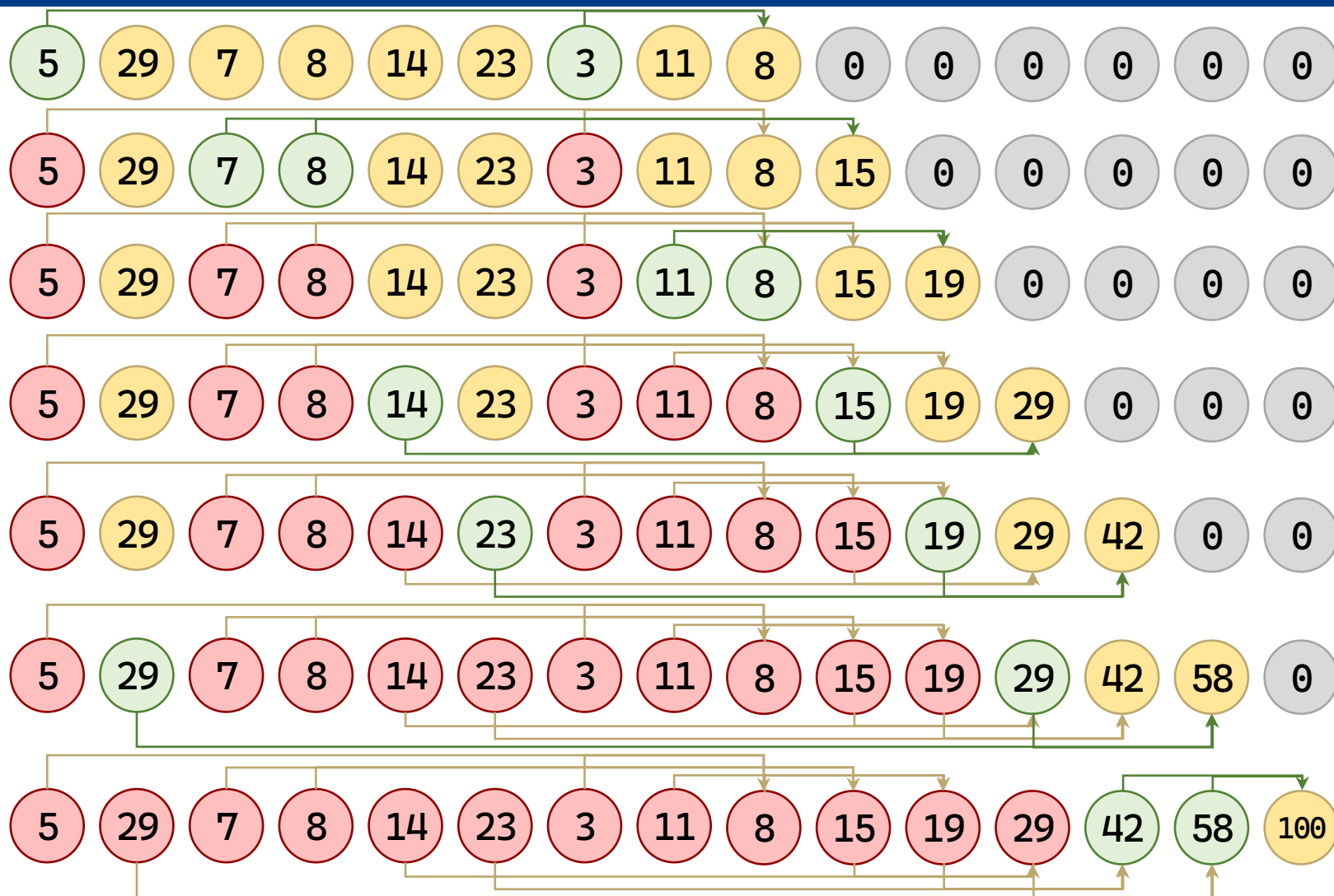


```

int main() {
    int weights[MAX_LEAVES];
    char huffmanCodes[MAX_LEAVES][MAX_LEAVES];
    printf("请输入叶子节点个数 n (1~%d): ", MAX_LEAVES);
    scanf("%d", &n);
    if (n < 1 || n > MAX_LEAVES) {
        printf("输入不合法! \n"); return -1;
    }
    printf("请输入 %d 个权值 (空格分隔) : ", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &weights[i]);
    createHuffmanTree(weights);
    getHuffmanCodes(huffmanCodes);
    printf("\n===== 哈夫曼编码表 =====\n");
    for (int i = 0; i < n; i++)
        printf("权值 %2d 的编码: %s\n", weights[i], huffmanCodes[i]);
    int wpl = 0;
    for (int i = 1; i <= n; i++) {
        int len = strlen(huffmanCodes[i - 1]);
        wpl += htree[i].weight * len;
    }
    printf("该哈夫曼树的带权路径长度 WPL = %d\n", wpl);
    return 0;
}

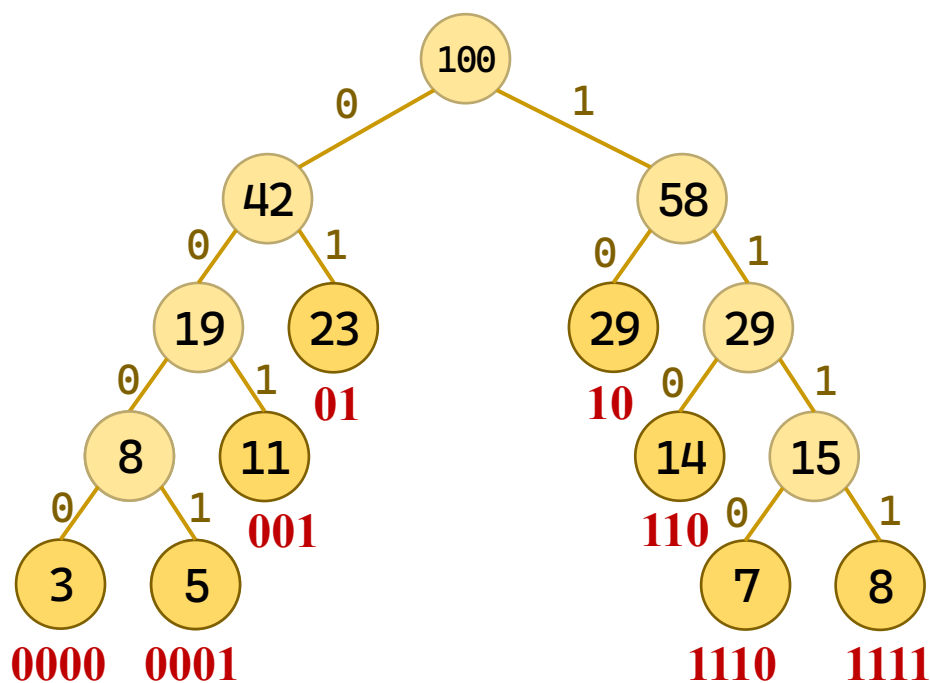
```

哈夫曼树的实现



哈夫曼树的实现

$$\begin{aligned} W &= \sum_i w_i \cdot l_i \\ &= 3 \times 4 + 5 \times 4 + 11 \times 3 + 23 \times 2 + 29 \times 2 + 14 \times 3 + 7 \times 4 + 8 \times 4 \\ &= 12 + 20 + 33 + 46 + 58 + 42 + 28 + 32 \\ &= 271 \end{aligned}$$



哈夫曼树的实现

请输入叶子节点个数 n (1~20): 8

请输入权值 (空格分隔) : 5 29 7 8 14 23 3 11

>>>>> 开始合并过程 <<<<<

第1次合并:选[7]权=3和[1]权=5, 建[9]权=8

第2次合并:选[3]权=7和[4]权=8, 建[10]权=15

第3次合并:选[9]权=8和[8]权=11, 建[11]权=19

第4次合并:选[5]权=14和[10]权=15, 建[12]权=29

第5次合并:选[11]权=19和[6]权=23, 建[13]权=42

第6次合并:选[2]权=29和[12]权=29, 建[14]权=58

第7次合并:选[13]权=42和[14]权=58, 建[15]权=100

===== 哈夫曼数组存储表 =====

下标	权值	父节点	左孩子	右孩子
1	5	9	0	0
2	29	14	0	0
3	7	10	0	0
4	8	10	0	0
5	14	12	0	0
6	23	13	0	0
7	3	9	0	0
8	11	11	0	0

9	8	11	7	1
10	15	12	3	4
11	19	13	9	8
12	29	14	5	10
13	42	15	11	6
14	58	15	2	12
15	100	0	13	14

=====

===== 哈夫曼编码表 =====

权值 5 的编码: 0001

权值 29 的编码: 10

权值 7 的编码: 1110

权值 8 的编码: 1111

权值 14 的编码: 110

权值 23 的编码: 01

权值 3 的编码: 0000

权值 11 的编码: 001

该哈夫曼树的带权路径长度 WPL = 271

目录

	2	二叉树
	3	遍历二叉树和线索二叉树
	4	树和森林
	5	哈夫曼树及其应用
	6	小结

本章小结

- 树的定义、满二叉树、完全二叉树。
- 二叉树性质。
- 二叉树存储结构：
 - (1) 顺序存储结构(适用于完全二叉树)。
 - (2) 链式存储结构及其基本操作。
- 二叉树遍历递归算法(3个)
- 二叉树遍历非递归算法(3个)

本章小结

- 先序后继线索化算法
- 树的存储结构及其应用
- 子集树
- 排列树
- 哈夫曼树、哈夫曼编码

本章小结

例3-11 编写递归算法：对于二叉树中每一个元素值为x的结点，删去以它为根的子树，并释放相应的空间。

```
void freeTree(BiTree T) {
    if (T == NULL) return;
    freeTree(T->lchild); freeTree(T->rchild);
    free(T);
}

BiTree deleteSubtree(BiTree T, int x) {
    if (T == NULL) return NULL;
    if (T->data == x) {
        freeTree(T); return NULL;
    }
    T->lchild = deleteSubtree(T->lchild, x);
    T->rchild = deleteSubtree(T->rchild, x);
    return T;
}
```


本章小结

- 例3-12 求二叉链表T中值为x的数据元素所在的层次。

```
int findLevelRecur(BiTree T, int x, int level) {  
    if (T == NULL) return 0;           // 空树未找到  
    if (T->data == x) return level;     // 找到, 返回当前层次  
    // 先查左子树  
    int leftRes = findLevelRecur(T->lchild, x, level + 1);  
    if (leftRes != 0) return leftRes;   // 左子树找到了, 直接返回  
    // 左子树没找到, 再查右子树  
    return findLevelRecur(T->rchild, x, level + 1);  
}  
  
int getLevel(BiTree T, int x) {        // 对外接口: 求值为x的结点的层次  
    return findLevelRecur(T, x, 1);    // 根结点层次为1  
}
```

本章小结

• 例3-13 判断二叉树是否相似。

```
bool isSimilar(BiTree T1, BiTree T2) {  
    // 情况1: 两棵树都为空 → 形状相同 (相似)  
    if (T1 == NULL && T2 == NULL) {  
        return true;  
    }  
    // 情况2: 一棵为空, 另一棵不为空 → 形状不同 (不相似)  
    if (T1 == NULL || T2 == NULL) {  
        return false;  
    }  
    // 情况3: 两棵都不为空 → 递归判断左子树和右子树是否都相似  
    // 注意: 这里完全没有比较 T1->data 和 T2->data, 只比较结构!  
    return isSimilar(T1->lchild, T2->lchild) &&  
        isSimilar(T1->rchild, T2->rchild);  
}
```

谢谢观看



廈門大學
XIAMEN UNIVERSITY



信息学院 黄 焯
(特色化示范性软件学院) 博士, 副教授
School of Informatics Wei Huang